Open MCEconomic API

SUM Server Setup Guide

Applies to	SUM (Server Utility Mod), Minecraft 1.12.2
Protocol	Open MCEconomic API 1.0
Companion	`OPEN_MCECONOMIC_API_SPECIFICATION`

Connecting SUM to a remote economy service is optional. With nothing configured, SUM runs a complete economy of its own out of the world save — wallets, bank accounts, ATMs and all. This guide covers what changes when you point it at a service instead, and what each setting protects you from.

Contents

1 Decide whether you need this **2**

2 The minimum viable configuration **2**

3 Security settings **3**
3.1 HTTPS is required . . . 3
3.2 Self-signed and private-CA certificates . . . 3
3.3 Where the token may live . . . 4
3.4 Optional request signing . . . 4

4 Single-player and LAN worlds **5**

5 What happens when the service is down **5**

6 Tuning **5**

7 Integrity headers **6**

8 Administration **6**
8.1 Two balances, so commands name one . . . 6
8.2 Player accounts . . . 7

9 Troubleshooting **7**

10 Before going live **7**

1 Decide whether you need this

SUM's economy works with no service at all. Before configuring anything, it is worth being clear about what a remote service actually buys you.

Without a service	With one
Bank accounts live in the world save.	Bank accounts live outside Minecraft, and can be shared with other systems (a Discord bot, a website, another server).
Balances are per-world. Copy the save, copy the money.	One balance follows the player everywhere, including out of the game.
Nothing to run, nothing to secure.	An HTTP service to host, a token to protect, TLS to terminate.

Wallets are never remote. Whatever you configure, the money players carry — their wallet balance and the bills in their inventory — stays local to the world save. Shops, plots, job escrow and `/pay` all spend it, and none of them ever touch the network. Only the **bank account** can be owned by a service, and the ATM is the only place the two meet.

That boundary is the single most important thing to understand before configuring anything: a service outage cannot stop players trading, and a laggy connection cannot slow down a shop.

2 The minimum viable configuration

The settings live in the `economy_api` category of `config/sum.cfg`, generated on first launch, and the token lives in `config/sum-secrets.cfg`, which a dedicated server creates on its first start with the economy API enabled. Four settings turn it on:

```
# config/sum.cfg -- safe to ship in a modpack
economy_api {
    B:enabled=true
    S:baseUrl=https://economy.example.net
    S:instanceId=alto-main
}

# config/sum-secrets.cfg -- server only, never shipped
economy_api {
    S:authToken=<the token your service operator gave you>
}
```

`authToken` can still be set in `sum.cfg` itself, as before, but SUM warns about it at startup; see Section 3.3.

Setting	Notes
`enabled`	Master switch. False by default, so an unconfigured server behaves exactly as it always has.
`baseUrl`	Root URL **without** the `/api/economic/v1` suffix — SUM appends that. Must be `https` unless you deliberately opt out (Section 3).
`authToken`	Issued by whoever runs the service. **A secret.**
`instanceId`	Identifies this server on every ledger entry. Give each server sharing a service a distinct value, so you can tell later which one a transaction came from.

Restart the server. On success the log reads:

```
[omce] Connected to <service> - currency BUCKS ($, 0 decimal place(s)), status ok.
[omce] Capabilities: void, events, headerPolicy, autoProvisionAccounts, offlinePlayers
```

If a capability the service lacks changes SUM's behaviour, it says so on the same startup — for example `Service has no 'fees' capability`, or a warning that a coarse currency will round prices. Those lines are informational, not errors.

3 Security settings

3.1 HTTPS is required

SUM refuses a plaintext `baseUrl`. Every request carries the bearer token in a header and every response carries account balances; over plain HTTP both are readable and forgeable by anything on the path.

`B:enableHttp=true` exists for local development only — pointing at `http://127.0.0.1` while testing. It logs a prominent warning on *every* server start while enabled, deliberately. Never set it on a server players use.

3.2 Self-signed and private-CA certificates

There is no switch to disable certificate verification, and adding one would defeat the point of having it. Instead, supply the certificate:

```
S:certificatePath=economy-ca.pem
S:certificatePassword=          # only for .p12/.pfx/.jks, not PEM
```

The path is resolved **relative to the config file**, so dropping the certificate next to `sum.cfg` and naming it is enough. Absolute paths work too.

Format	Extensions	Notes
PEM	`.pem`, `.crt`, `.cer`	Preferred. One or more certificates.
PKCS#12	`.p12`, `.pfx`	Needs `certificatePassword`.
Java KeyStore	`.jks`	Needs `certificatePassword`.

The supplied certificate is added *alongside* the system trust store rather than replacing it, so a private CA can coexist with public ones. Hostname verification still applies. An unreadable,

malformed or expired certificate stops the economy backend from starting at all — that is deliberate, because the alternative is silently falling back to an unverified connection.

3.3 Where the token may live

Never ship `authToken` in a modpack. Config files distributed to players are readable by them, and a token in one is a published token. Anyone holding it can move money.

A Minecraft *client* never contacts the service — it receives balances from its server over SUM's own sync packet — so a pack config needs none of the three secrets (`authToken`, `hmacSecret`, `certificatePassword`). SUM reads each one from the first of these that holds a non-blank value:

1. An environment variable, which is convenient for containerised servers:
   - `SUM_ECONOMY_AUTH_TOKEN`
   - `SUM_ECONOMY_HMAC_SECRET`
   - `SUM_ECONOMY_CERT_PASSWORD`
2. The secrets file, `config/sum-secrets.cfg`. It is an ordinary Forge config with an `economy_api` category holding the keys `S:authToken`, `S:hmacSecret` and `S:certificatePassword`. To move it, point `economy_api.secretsFile` in `sum.cfg` at another name or an absolute path.
3. `sum.cfg` itself, for servers set up before the secrets file existed.

This is what lets a launcher or modpack manage `sum.cfg` on the server as well: ship `sum.cfg` with `enabled`, `baseUrl` and `instanceId`, and put the token in `sum-secrets.cfg` on the server by hand. Pack tooling never touches that file. A dedicated server with the economy API enabled writes an empty, commented `sum-secrets.cfg` on first start (owner-only where the filesystem supports it); a client never creates one.

Each start logs where every secret came from, without its value:

```
[omce] Economy secrets: authToken <set> (from sum-secrets.cfg), hmacSecret <unset> (from unset)
    ,
       certificatePassword <unset> (from unset).
```

A token still sitting in `sum.cfg` adds a warning asking you to move it.

Use a different token per server. If one is compromised you can revoke it without disturbing the others.

3.4 Optional request signing

```
S:hmacSecret=<shared secret>
```

Adds an HMAC-SHA256 signature to every request. The token proves *who* is calling; the signature proves the request was not tampered with or replayed. Only set it if your service verifies signatures — a service that ignores them is no worse off, but a service that requires them will reject you without this.

4 Single-player and LAN worlds

Opening a single-player world starts a full *integrated server* inside your game client. From SUM's point of view that is a server, and without a guard it would connect to the shared economy and spend real balances out of a local save.

`B:allowIntegratedServer` is `false` by default, so this cannot happen by accident. With it off, a single-player or LAN world silently falls back to SUM's local economy — everything still works, it is just a separate economy belonging to that save.

Set it `true` only when you deliberately want a local world to transact against the live economy, for instance while testing an integration. Even then SUM still reports itself honestly as an integrated server, so the service can refuse it on its own terms.

5 What happens when the service is down

`S:unavailablePolicy` decides. The default is the conservative one.

Value	Behaviour
`deny` *(default)*	Cached balances are shown and marked stale; all bank spending is refused with a clear message. **No money moves without the authority.**
`cached`	As above, but transactions are queued in memory and flushed on recovery, up to a bounded queue. Beyond that bound it behaves as `deny`.
`local`	Falls back to SUM's own bank balances. **These will diverge from the service** and need manual reconciliation afterwards. Intended for single-player and offline testing, not for a live server.

Remember that only *banking* is affected. Shops, plots, job escrow and `/pay` spend the local wallet and keep working normally through any outage.

6 Tuning

The defaults are sensible for a server of any size; these are worth changing only if you have a reason.

Setting	Default	Raise or lower it when…
`balanceCacheTtlSeconds`	30	Balances look stale in game. Lower it for fresher figures at the cost of more requests. In-game reads always hit this cache, never the network, so this never affects tick time.
`eventPollSeconds`	10	Changes made outside Minecraft take too long to appear. `0` disables the feed and relies on the periodic refresh. Ignored if the service lacks the `events` capability.
`healthPollSeconds`	60	You want faster detection of the service coming back after an outage.

Setting	Default	Raise or lower it when. . .
`connectTimeoutMs`	3000	The service is geographically distant. Nothing here runs on the server thread, so a longer timeout costs latency, never TPS.
`readTimeoutMs`	5000	As above.
`maxRetries`	2	Retries reuse the same idempotency key, so a retried payment can never double-charge. Raising it is safe; lowering it makes transient failures more visible to players.
`verboseLogging`	false	Diagnosing an integration. Logs every request and response. The token, HMAC secret and signature are never logged regardless.

7 Integrity headers

`B:sendIntegrityHeaders` (default `true`) sends a small amount of context a service can use for stricter policy: whether this server runs in online mode, a per-world id and a monotonic counter, a per-boot session id, the online player count, risk-relevant state of the acting player (creative, op, cheats), and the mod version.

> The world id and counter are the useful pair. The counter lives *in the world save*, so restoring a backup takes it back down with it — a service seeing a counter it has already passed knows in-world state was rolled back while its ledger was not. That is the classic duplication exploit, and it is the one signal here a service can verify rather than simply believe.

Nothing invasive is sent, and a conforming service must work without any of it:

- No IP addresses, hostnames or network identifiers.
- No hardware or installation fingerprints.
- No mod list, no world seed, no world contents.
- No player identities other than the parties to the transaction.

Set it `false` to omit them. A service that *requires* one will then refuse you, and will say which header it wanted.

8 Administration

8.1 Two balances, so commands name one

```
/sum econ balance [player]          # shows wallet and bank
/sum econ add <amount> [player]         # wallet (default)
/sum econ add <amount> [player] --bank  # bank account
/sum econ set <amount> [player] [--bank]
```

Wallet is the default because it is the balance players spend.

Bank adjustments are **refused** while a remote service owns the account. That ledger is not SUM's to write to directly; make the correction on the service side, where it can be recorded properly.

8.2 Player accounts

How a Minecraft UUID becomes an account is the service's business, not SUM's. A service that requires linking returns an instruction, which SUM relays verbatim into chat — so a player who is not linked is told exactly what to do by the service that needs it.

9 Troubleshooting

Symptom	Cause and fix
Nothing in the log at all	`enabled` is false, or the config was not saved. The backend starts when a *world* loads, not at game launch.
`economy_api.baseUrl uses plaintext http://`	Working as intended. Use `https`, supply a certificate via `certificatePath`, or set `enableHttp` for local testing only.
`Economy API is configured, but this is an integrated server`	A single-player or LAN world. Set `allowIntegratedServer` if that is genuinely what you want.
`Economy API disabled -- could not load certificatePath`	Wrong path (it is relative to the config file), wrong password, or an expired certificate. Deliberately fatal rather than falling back to an unverified connection.
ATM says *Bank unavailable*	The service has not resolved an account for that player yet. Usually means unlinked — check the log for `ACCOUNT_NOT_LINKED` and the service's linking instruction.
`ENVIRONMENT_REJECTED`	The service refuses this kind of server: commonly an offline-mode or single-player one. `details.reason` names which.
Balances look stale	Lower `balanceCacheTtlSeconds`, or check whether the service advertises `events` — without it, outside changes only appear on the periodic refresh.
Prices rounded up in game	The service settles in whole units. Expected; see the specification's conversion section.

`verboseLogging=true` logs every request and response, which is usually enough to see what a service is objecting to. It never logs the token or signature, so the output is safe to paste when asking for help.

10 Before going live

☐ `baseUrl` is `https`, and `enableHttp` is `false`.

☐ `authToken` is set on the **server only**, in `sum-secrets.cfg` or an environment variable, and is not in any config shipped to players. The startup log line `Economy secrets: authToken <set> (from ...)` names where it came from.

☐ This server has its own token, not one shared with another server.

☐ `instanceId` is distinct from every other server using the same service.

☐ `allowIntegratedServer` is `false` unless you specifically want local worlds transacting.

☐ `unavailablePolicy` is `deny` or `cached` — `local` will diverge.

☐ A player has actually deposited and withdrawn at an ATM, and the amounts match on both sides.